

Anwendungsorientierte Softwareentwicklung

Laborblatt 9: Mehr Kontrolle!

Lernziele

- **Hardware (Taster & Potentiometer) per serieller Schnittstelle an ein Spiel anbinden**
- **Bestehenden Mikrocontroller-Code verstehen und wiederverwenden**
- **Verbindung zwischen Spiel und Controller herstellen**
- **Wie man ein "richtiges" Modul schreibt**

Rückblick und Ziel für heute

Bisher habt ihr am Breakout-Spiel gearbeitet, das mit der Tastatur gesteuert wurde. Heute erweitern wir es – und zwar mit einem selbstgebauten **Game-Controller auf dem Breadboard!**

In **Laborblatt 4 und 5** habt ihr einen Mikrocontroller programmiert, der

- den **Taster-Zustand** erkennt (gedrückt / losgelassen)
- den **Analogwert** des Potentiometers misst (ein **10-Bit-Wert**, also eine ganze Zahl aus dem Bereich **von 0 bis 1023**)

Diese Informationen werden über die serielle Schnittstelle gesendet – zum Beispiel:

- **Taster gedrückt**: eine Zeile mit `A`
- **Taster losgelassen**: eine Zeile mit `a`
- **Spannungsänderung am Potentiometer**: eine Zeile wie `X 528`

Diese Daten könnt ihr jetzt direkt ins Spiel einbauen:

- Der **Taster** feuert Raketen ab
- Das **Potentiometer** steuert die **Position des Paddles**

Vorbereitung

Wie üblich arbeiten wir mit zwei Terminals:

- **links**: Quellcode editieren
- **rechts**: Code kompilieren, flashen, seriellen Monitor öffnen, Spiel testen

Repository klonen

Im linken oder rechten Terminal:

```
git clone https://gitlab.uni-ulm.de/BENUTZERNAME/asp-ulm
```

Dann in beiden Terminals ins Verzeichnis mit dem Mikrocontroller-Code wechseln:

```
cd asp-ulm/controller
```

Im nächsten Schritt sehen wir uns den vorhandenen Code nochmal genauer an:

- Wie funktioniert das Makefile?
- Wie wird der Code kompiliert und auf den Mikrocontroller übertragen?
- Wie öffnet man den seriellen Monitor, um die gesendeten Daten zu sehen?

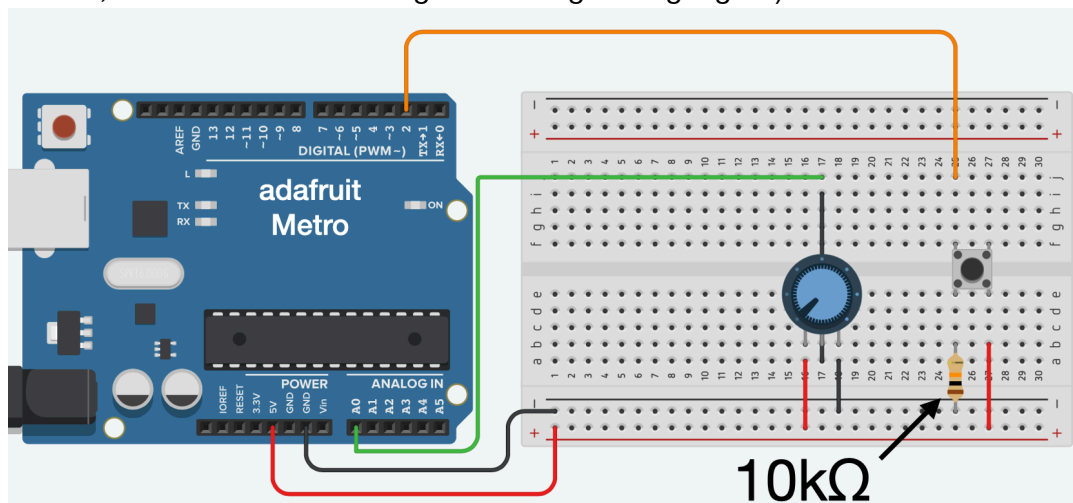
Schritt 1: Mikrocontroller-Code und Breadboard-Bastelei

Öffnet im **linken Terminal** die Datei `controller.ino` mit dem C++-Code für den Mikrocontroller. Geht den Code noch einmal in Ruhe durch und versucht nachzuvollziehen, was genau passiert. Falls euch etwas unklar ist, könnt ihr natürlich

- nochmal einen Blick in Laborblatt 4 werfen,
- uns fragen,
- oder ihr helft euch gegenseitig auf die Sprünge.

Wir stellen übrigens auch gerne mal dumme Fragen – das gehört dazu 😊

Bevor ihr den Mikrocontroller-Code ausführt, solltet ihr auf dem **Breadboard** den **bekannten Schaltkreis** mit **Taster und Potentiometer** wieder aufbauen und korrekt an den Mikrocontroller anschließen. Hier ein Bild mit dem vollständigen Aufbau (der Taster ist am digitalen Eingang **Pin 2** angeschlossen, das Potentiometer hängt am analogen Eingang **A0**):



Im aktuellen Verzeichnis `controller` befindet sich auch eine Datei namens `Makefile`. Wenn ihr möchtet, könnt ihr euch den (etwas kryptischen) Inhalt im **rechten Terminal** mit `cat Makefile` kurz anzeigen lassen. Für heute reicht es aber zu wissen, **wofür dieses Makefile gut ist**. Es enthält Regeln, um automatisch die passenden Werkzeuge aufzurufen – und zwar für:

- die **Übersetzung** des C++-Codes (`controller.ino`) in **Maschinencode** für den Mikrocontroller
- das Flashen des Mikrocontrollers mit `avrdude`
- das Öffnen eines seriellen Monitors, um die Ausgaben des Controllers zu sehen

Das Programm `make` übernimmt all das für euch – je nach aufgerufenem Ziel:

Befehl	Bedeutung
<code>make</code>	Übersetzt den Code in Maschinencode (Hex-Datei)
<code>make upload</code>	Flasht den Mikrocontroller (kompiliert vorher, falls nötig)
<code>make monitor</code>	Öffnet den seriellen Monitor, der anzeigt, was der Mikrocontroller sendet

⚠ Wichtig: Damit `make upload` oder `make monitor` funktioniert, muss der **Mikrocontroller angeschlossen** sein und der **USB-Port unter WSL sichtbar gemacht** werden – das geht wie gewohnt mit: `usb_check`

Aufgabe: Ist alles aufgebaut und der Mikrocontroller angeschlossen, übersetzt das Programm, flasht den Mikrocontroller und prüft mit dem seriellen Monitor, ob alles wie erwartet funktioniert.

Schritt 2: Serielle Schnittstelle mit Python auslesen (`serial-reader.py`)

Wir haben übrigens auch unseren eigenen seriellen Monitor in Python geschrieben (siehe Laborblatt 4). Den schauen wir uns jetzt nochmal an – diesmal mit frischem Blick!

Vorbereitung

Wechselt in beiden Terminals ins Nachbarverzeichnis `python`:

```
cd ../python
```

Öffnet im **linken Terminal** die Datei `serial-reader.py`.

Im **rechten Terminal** könnt ihr den Monitor starten mit:

```
python serial-reader.py
```

Aufgabe

Als wir **Laborblatt 4** bearbeitet haben, kannten wir uns mit objektorientierter Programmierung noch nicht besonders aus. Das ist jetzt anders – und genau deshalb lohnt sich ein zweiter Blick!

Geht den Code `serial-reader.py` von Schritt für Schritt durch – und bearbeitet folgende Aufgaben:

1. Objektorientierte Schnittstelle zur seriellen Verbindung

```
ser = serial.Serial("/dev/ttyUSB0", 9600)
```

Hier wird ein Objekt der Klasse `Serial` aus dem Modul `serial` erzeugt. Dieses Objekt (das wir `ser` nennen) verwaltet:

- die serielle Schnittstelle,
- einen (internen) Puffer für empfangene Bytes,
- und stellt Methoden zur Verfügung, um damit zu arbeiten.

Wichtige Methoden:

- `in_waiting`: Gibt an, **wie viele Bytes aktuell im Puffer liegen**
 - `read(n)`: Liest **n Bytes aus dem Puffer** (FIFO – First In, First Out)
- In unserem Code lesen wir **immer ein einzelnes Byte** mit `ser.read(1)`.

2. Kleine Codeänderung

Im Code findet ihr diese Zeile:

```
ch = bytes.decode(ch)
```

Ändert sie in:

```
ch = ch.decode()
```

Dann startet das Programm erneut und überprüft, ob noch alles wie zuvor funktioniert. Diskutiert oder fragt uns, warum die neue Zeile äquivalent zur alten ist.

Hinweis: `ser.read(1)` gibt ein Objekt vom Typ `bytes` zurück.

Wenn ihr fertig seid, macht einen Commit, damit die kleine Änderung am Code nicht verloren geht:

```
git commit -a -m "decode als Methode aufrufen"
git push
```

Schritt 3: Python Grundlagen (Strings aufteilen – Zahlen umwandeln)

Bevor wir gleich die seriellen Daten ins Spiel einbauen, brauchen wir noch ein paar praktische Grundlagen in Python.

Das Problem: Die serielle Schnittstelle liefert uns Zeichenketten wie "X 512". Praktischer wäre es, wenn wir daraus direkt eine Liste wie ["X", 523] machen könnten – also: Den Text beim Leerzeichen aufteilen und den zweiten Teil in eine Zahl umwandeln.

Kleine Python-Session

Startet im rechten Terminal die Python-Shell mit: `python`. Dann geht's los:

1. Gebt ein:

```
>>> s = "X 512"
```

2. Jetzt teilt ihr den String auf:

```
>>> l = s.split()
```

3. Prüft den Inhalt:

```
>>> s
```

```
>>> l
```

→ `l` sollte ["X", "523"] enthalten.

Probiert auch mal:

```
>>> s = "X 512"
```

```
>>> l = s.split()
```

Mehrere Leerzeichen machen keinen Unterschied – `split()` ignoriert überflüssige Leerzeichen.

4. Greift auf einzelne Listenelemente zu:

```
>>> l[0]
```

```
>>> l[1]
```

5. Was passiert hier?

```
>>> l[1] + 1
```

Das gibt einen Fehler – denn `l[1]` ist ein String, keine Zahl.

6. Umwandeln den String um in eine Zahl und versucht es nochmals:

```
>>> l[1] = int(l[1])
```

```
>>> l[1] + 1
```

7. Wir könnten auch Strings mit mehr als eine Zahl verarbeiten – zum Beispiel:

```
>>> s = "X 523 23"
```

```
>>> l = s.split()
```

```
>>> for i in range(1, len(l)):
```

```
...     l[i] = int(l[i])
```

→ Lasst euch danach `l` anzeigen.

Ihr seht: Ab dem zweiten Element sind alle Einträge in ganze Zahlen umgewandelt worden.

Warum das nützlich ist

Normalerweise wollen wir ja nur das Notwendige programmieren – keine Verallgemeinerungen auf Vorrat. Aber hier brauchen wir diese Verallgemeinerung tatsächlich – nicht wegen Fällen mit mehreren Zahlen, sondern wegen Fällen mit keiner Zahl:

```
>>> s = "A"
```

```
>>> l = s.split()
```

```
>>> for i in range(1, len(l)):
```

```
...     l[i] = int(l[i])
```

Das funktioniert ebenfalls – weil die Schleife nicht durchlaufen wird, wenn `l` nur ein Element hat.

Schritt 4: Unser Python-Interface für die serielle Schnittstelle (`controller.py`)

Öffnet im **linken Terminal** die Datei `controller.py` (die wurde in Laborblatt 5 erstellt) und startet sie im **rechten Terminal** mit: `python controller.py`

Ist das objektorientiert?

Obwohl im Code eine Klasse `Controller` auftaucht, handelt es sich **nicht** um eine objektorientierte Schnittstelle – was bei hardwarenahen Schnittstellen durchaus üblich ist.

Stattdessen gibt es **zwei freie Funktionen**, die so verwendet werden können:

```
connect("/dev/ttyUSB0")  
line = readline()
```

- `connect(port)` muss vorher aufgerufen werden, um die Verbindung aufzubauen.
- `readline()` gibt entweder:
 - `None` zurück (wenn noch keine vollständige Zeile vorliegt), oder
 - den **empfangenen String** (wenn eine Zeile mit `\n` abgeschlossen wurde).

Was macht die Klasse eigentlich?

Die Klasse `Controller` enthält zwei statische Attribute:

- `Controller.ser` – die serielle Verbindung
- `Controller.buffer` – der aktuelle Eingabepuffer

Diese dienen als **globale Variablen**, sind aber durch die Klasse **klar gekapselt** – das zeigt, dass sie logisch zusammengehören.

Die beiden Funktionen `connect()` und `readline()` greifen auf diese Attribute zu – man könnte sie also auch direkt in die Klasse als statische Methoden verschieben und dann so aufrufen:

```
Controller.connect(port)  
Controller.readline()
```

Das **müsst ihr nicht** tun – aber ihr **dürft**, wenn ihr Lust habt, das einmal auszuprobieren.

Aufgabe

1. Schreibt jetzt eine Funktion:

```
def readdata():
```

Diese soll versuchen mit `readline()` eine Zeile einzulesen. Falls das erfolgreich war (man also nicht `None` erhalten hat):

- die Zeile wie in **Schritt 3** in eine Liste aufteilen, also **alle Zahlen ab dem zweiten Element** mit `int()` umwandeln,
- und die resultierende Liste zurückgeben.

Wenn keine Zeile vorliegt (`readline()` gab `None` zurück), soll auch `readdata()` den Wert `None` zurückgeben.

Zum Testen könnt ihr im Programm einfach in der Hauptschleife

```
line = readline()
```

durch

```
line = readdata()
```

ersetzen. Startet das Programm neu und überprüft, ob alles wie erwartet funktioniert – also: statt `"A"` oder `"a"` kommt `['A']` bzw. `['a']`, und bei `"X 523"` kommt `['X', 523]`.

2. Wenn alles klappt:

```
git commit -a -m "serial parser: readdata"  
git push
```

Schritt 5: `controller.py` in ein Modul umwandeln

Damit wir unseren Code später von anderen Python-Programmen aus verwenden können (z. B. aus dem Spiel heraus), machen wir aus `controller.py` ein richtiges Modul.

Dazu verschieben wir den bisherigen Code aus der Hauptschleife in eine Funktion `main()` und fügen am Ende der Datei folgendes hinzu:

```
if __name__ == "__main__":
    main()
```

Was bringt das?

- Wenn ihr `controller.py` **direkt startet** mit `python controller.py` wird wie bisher die `main()`-Funktion aufgerufen und das Programm läuft. Man kann das Modul also weiterhin **testen**.
- Wenn ihr das Modul **importiert**, z. B. mit `import controller` wird **nichts automatisch ausgeführt**. Man kann die Funktionalität also gezielt an anderer Stelle einbauen und verwenden. Genau das machen wir im nächsten Schritt!

Aufgabe

Testet kurz, ob beim Aufruf mit `python controller.py` noch alles wie gewohnt läuft. Wenn ja:

```
git commit -a -m "turned to Modul"
git push
```

Schritt 6: Mit dem Taster Raketen abfeuern 🚀

Jetzt wollen wir testen, wie man das Modul `controller.py` in unser Spiel einbaut – am Beispiel: **Der Taster feuert eine Rakete ab.**

1. Modul importieren

Öffnet im **linken Terminal** die Datei `breakout.py`. Importiert oben im Code das Modul mit

```
import controller
```

Falls ihr `connect`, `readline` und `readdata` als statische Methode innerhalb der Klasse `Controller` umgesetzt habt, verwendet: `from controller import Controller` sonst müsste ihr `controller.Controller.connect` statt `Controller.connect` schreiben.

2. Verbindung zur seriellen Schnittstelle aufbauen

Fügt vor der Hauptschleife die folgende Zeile ein:

```
controller.connect("/dev/ttyUSB0")
```

Falls ihr `connect` als statische Methode innerhalb der Klasse `Controller` umgesetzt habt, verwendet: `Controller.connect("/dev/ttyUSB0")`

3. Serielle Daten in der Spielschleife verarbeiten

Fügt in der Hauptschleife den folgenden Code ein:

```
ser_data = controller.readdata()
if ser_data:
    if ser_data[0] == "A":
        rockets.append(Rocket(Paddle.x + Paddle.w // 2, Paddle.y, -10))
```

Auch hier gilt: Wenn ihr `readdata()` als statische Methode in der Klasse habt, dann verwendet `ser_data = Controller.readdata()`

Es wird geprüft, ob eine neue Zeile von der seriellen Schnittstelle empfangen wurde. Wenn die erste Komponente `"A"` ist (Taster gedrückt), wird eine Rakete erzeugt und abgefeuert.

Testet euer Spiel. Wenn alles klappt:

```
git commit -a -m "taster feuert"
git push
```

Schritt 7: Mit dem Potentiometer das Paddle steuern

Jetzt fehlt (vorerst) nur noch eins: **Das Paddle direkt mit dem Potentiometer steuern.**

Dazu erweitern wir die Klasse Paddle um eine Methode `set()`, mit der man direkt die `x`-Position setzen kann.

1. Code in `breakout.py` erweitern

Ändert den Code zur Abfrage des Controllers zu:

```
ser_data = controller.readdata()
if ser_data:
    if ser_data[0] == "A":
        rockets.append(Rocket(Paddle.x + Paddle.w // 2, Paddle.y, -10))
    elif ser_data[0] == "X":
        Paddle.set(ser_data[1])
```

2. Methode `set()` in `paddle.py` schreiben

Öffnet die Datei `paddle.py` und ergänzt die Methode:

```
def set(x):
    Paddle.x = x
    if Paddle.x < 0:
        Paddle.x = 0
    if Paddle.x + Paddle.w > Game.width:
        Paddle.x = Game.width - Paddle.w
```

Damit wird `Paddle.x` gesetzt und bei Bedarf nach links und rechts begrenzt.

3. Testen und Feintuning

Startet das Spiel und testet, ob ihr das Paddle mit dem Potentiometer bewegen könnt.

Wenn es sich zu träge anfühlt, probiert in `breakout.py`:

```
Paddle.set(ser_data[1] * 1.5)
oder
Paddle.set(ser_data[1] * 2)
```

Wenn alles läuft (und ihr mit der Paddle-Geschwindigkeit zufrieden seid):

```
git commit -a -m "Paddle tut auch"
git push
```

Motivation für den nächsten Schritt: Es geht um harte Euros

Zieht jetzt testweise mal das USB-Kabel vom Mikrocontroller ab und startet das Spiel neu.

 Das Spiel stürzt ab, weil der Aufruf `controller.connect("/dev/ttyUSB0")` fehlschlägt. Stellt euch vor: Ihr habt ein großartiges Spiel programmiert, das schon ohne Controller Spaß macht – aber erst mit Controller wird es richtig cool. Und genau dafür sind die Leute bereit, ein paar harte Euros auszugeben.

 Damit das klappt, muss euer Spiel:

- **auch ohne Controller** problemlos laufen,
- und idealerweise sogar erlauben, den Controller **im laufenden Spiel anzuschließen**.

Schritt 8: Exceptions abfangen

Wir machen unseren Code jetzt robuster, indem wir Fehler beim Verbindungsaufbau sauber abfangen.

Aufgabe

1. `connect()` anpassen

Ändert in `controller.py` die Funktion `connect` so:

```
def connect(port):
    try:
        Controller.ser = serial.Serial(port, 9600)
        Controller.buffer = ""
        print(f"connected to {port}")
    except:
        print(f"could not connected to {port}")
```

Was passiert hier?

Ihr seht hier das Grundmuster:

```
try:
    ...
except:
    ...
```

Das hat folgende Bedeutung: Es wird versucht den Code im `try`-Block auszuführen. Falls dabei ein Fehler auftritt, springt das Programm in den `except`-Block. Wenn kein Fehler passiert, wird der `except`-Block einfach übersprungen.

In unserem Fall heißt das konkret:

- Wenn die serielle Schnittstelle sich öffnen lässt, ist alles gut – `Controller.ser` enthält ein gültiges Objekt.
- Wenn es fehlschlägt, gibt's nur eine Meldung, und `Controller.ser` bleibt auf `None` – das Programm stürzt (an dieser Stelle) nicht ab.

2. Testen und nächstes Problem lösen

Startet das Spiel **ohne angeschlossenen Mikrocontroller**. Ihr werdet sehen: Es stürzt trotzdem ab – aber erst später, nämlich wenn `readline()` aufgerufen wird. Warum? Weil diese Funktion annimmt, dass `Controller.ser` immer ein gültiges Objekt ist.

Das lässt sich leicht beheben. Wie müssen nur `readline()` absichern. Fragt in der Funktion `readline()` am Anfang ab, ob `Controller.ser` ein gültiges Objekt ist:

```
def readline():
    if Controller.ser is None:
        return None
    # Rest wie bisher
```

3. Testen und nächstes Problem lösen

Startet das Spiel jetzt bitte noch einmal, diesmal ohne den Mikrocontroller anzuschließen. Ihr werdet merken, dass es erneut abstürzt, aber diesmal an einer neuen Stelle, nämlich bei der Funktion `readdata()`.

Das liegt daran, dass wir vorher in `readline()` eine Abfrage eingebaut haben, die überprüft, ob `Controller.ser` überhaupt ein gültiges Objekt ist. Wenn das nicht der Fall ist, gibt `readline()` den Wert `None` zurück. Genau hier liegt das Problem: In der Funktion `readdata()` wird ohne weitere Prüfung sofort `s.split()` aufgerufen. Solange `s` ein String ist, funktioniert das problemlos, aber wenn `s` den Wert `None` hat, gibt es einen Absturz, weil ein `None`-Objekt keine `split()`-Methode besitzt.

Damit das nicht passiert, müssen wir `readdata()` so anpassen, dass zuerst geprüft wird, ob tatsächlich ein Wert vorhanden ist. Die Funktion könnte dann zum Beispiel so aussehen:

```
def readdata():
    s = readline()
    if s is not None:
        s = s.split()
        for i in range(1, len(s)):
            s[i] = int(s[i])
        return s
    return None
```

Jetzt sollte das Spiel auch ohne angeschlossenen Mikrocontroller laufen.

4. Test: Mikrocontroller im laufenden Betrieb anschließen

Ausnahmsweise wird jetzt das **linke Terminal nicht zum Editieren** verwendet, sondern um `usb_check` auszuführen – also unseren bekannten Workaround, damit WSL den USB-Port sieht. Wir wollen nämlich testen, ob das Spiel auch dann funktioniert, wenn der Mikrocontroller angeschlossen ist. Dabei unterscheiden wir zwei Fälle:

- (a) Der Mikrocontroller ist schon angeschlossen, bevor das Spiel startet, oder
- (b) er wird angeschlossen, während das Spiel bereits läuft.

Testet zuerst Fall (a): Schließt den Mikrocontroller an, führt im linken Terminal `usb_check` aus und startet dann im rechten Terminal das Spiel mit `python breakout.py`. Mit dem Breadboard solltet ihr jetzt das Spiel steuern können.

Als Nächstes Fall (b): Zieht das USB-Kabel vom Mikrocontroller ab, startet im rechten Terminal das Spiel erneut mit `python breakout.py`, schließt dann den Mikrocontroller wieder an und führt im linken Terminal `usb_check` aus.

Ihr werdet merken: **Leider funktioniert das Breadboard jetzt nicht mehr.** Der Grund ist einfach: Unser Programm versucht nur einmal am Anfang, mit

```
controller.connect("/dev/ttyUSB0")
```

die serielle Schnittstelle zu öffnen. Wenn das beim Start fehlschlägt, wird es später nicht nochmal versucht.

Eine einfache, aber praktische Lösung ist, dass man das Programm so erweitert, dass der Benutzer jederzeit durch Drücken der Taste **P** (für „Peripherie“) im laufenden Spiel versuchen kann, den Game-Controller zu aktivieren. Das ist zwar noch kein echtes Hot Plug’n’Play, aber es ist eine alltagstaugliche Lösung.

Und genau hier kommt ihr ins Spiel:

Passt euer Programm so an, dass das funktioniert. Wenn am Ende alles klappt, nicht vergessen:

```
git commit -a -m "plug and play"
git push
```

Ausblick: Was euch im nächsten Laborblatt erwartet

Der kniffligste Teil ist geschafft – Glückwunsch! Ab jetzt geht es nur noch darum, das Spiel weiter auszubauen und immer spannender zu machen. Und das erwartet euch beim nächsten Mal:

- Es wird Bricks geben, die robuster sind und mehrfach getroffen werden müssen.
- Manche Bricks haben überraschende Nebeneffekte: Bei manchen bekommt ihr einen Extraball, bei anderen tauchen plötzlich weitere Raumschiffe auf oder es passieren andere aufregende Dinge.
- Und die Raumschiffe? Die sind leider nicht mehr so harmlos wie bisher – sie beginnen, Bomben abzuwerfen. Zum Glück gibt es aber auch Bricks, die euch ein Schutzschild verschaffen.
- Und ... na ja, lasst euch überraschen!

Das Schöne: Viel Neues müsst ihr dafür nicht lernen. Das meiste wird darin bestehen, den bestehenden Code so umzuorganisieren, dass alles sauber zusammenpasst. Und genau das ist völlig normal beim Programmieren. Zu Beginn eines Projekts kann man eben nicht jedes Detail vorhersehen – und gerade das macht es spannend.

Wer selber experimentieren möchte

Wenn ihr Lust habt, schon vor dem nächsten Laborblatt eigene Features umzusetzen, hier ein paar Ideen, wie ihr vorgehen könnt. Wichtig dabei ist: Macht immer möglichst kleine Änderungen, die ihr sofort testen könnt. So kommt ihr Schritt für Schritt ans Ziel, ohne dass euch am Ende alles um die Ohren fliegt.

Globale Verwaltung von Bällen und Raumschiffen

Momentan werden die Listen `balls` und `ships` lokal in `breakout.py` verwaltet. Das bedeutet, nur dort kann man neue Bälle oder Raumschiffe ins Spiel bringen. Eine gute erste Übung wäre es, diese Listen als statische Attribute in die Klasse `Game` zu verschieben. Dann könnt ihr von anderen Modulen aus darauf zugreifen. Das ist eine kleine Änderung, die sich schnell umsetzen und sofort testen lässt.

Bricks mit unterschiedlichen Effekten

In `wall.py` gibt es die Matrix `Wall.brick`. Bisher hat jeder Eintrag nur den Wert `0` (kein Brick) oder `1` (normaler Brick). Ihr könntet das Initialisieren so erweitern, dass dort zufällig auch `2` vorkommt. In der Funktion `collision` könnt ihr dann abfragen, was für ein Brick getroffen wurde: Bei `1` bleibt alles wie bisher, bei `2` bringt ihr zusätzlich einen neuen Ball ins Spiel. Wenn das funktioniert, merkt ihr wahrscheinlich schnell: Man sieht den Bricks nicht an, was sie Besonderes tun. Das könnt ihr ändern, indem ihr die `draw`-Funktion anpasst.

Raumschiffe hinzufügen

Analog könnt ihr Bricks auch mit den Werten `0`, `1`, `2` oder `3` initialisieren. Bei einem Brick mit Wert `3` könnte zum Beispiel ein neues Raumschiff ins Spiel kommen.

Robustere Bricks

Ihr könnt den Wertebereich weiter ausbauen, sodass ein Brick mit `4` nach der ersten Kollision nicht verschwindet, sondern auf `1` zurückgesetzt wird. Damit muss man ihn zweimal treffen, um ihn zu

Bomben werfen

Bomben lassen sich ähnlich umsetzen wie Raketen. Ihr könntet eine Klasse `Bomb` schreiben, die analog zur Klasse `Rocket` aufgebaut ist. Eine Liste für die Bomben könntet ihr ebenfalls als statisches Attribut in der Klasse `Game` unterbringen, damit sie global sichtbar ist.

Wenn ihr das ausprobiert: Habt Spaß dabei und denkt daran, regelmäßig kleine Zwischenschritte zu testen! Das ist beim Programmieren oft der entscheidende Trick.

Und vergesst nicht, eure Fortschritte ins Git-Repository zu committen. Falls ihr neue Dateien anlegt, müsst ihr sie vorher einmalig mit `git add DATEINAME` registrieren – danach sind sie wie die anderen Dateien Teil eures Projekts.